Lý thuyết đồ thị

Cây và các thuật toán trên cây

Nhóm 7

Ngày 19 tháng 3 năm 2026

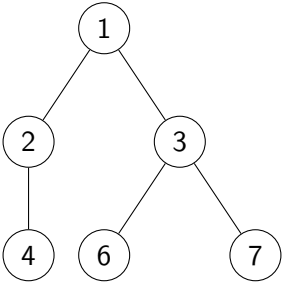
Nội dung

1. Định nghĩa cây
2. Ứng dụng của cây
3. Một số thuật toán trên cây
4. Kết luận

Cây là gì

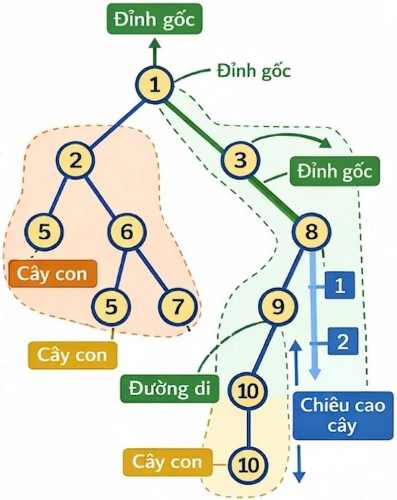
- Cây là một loại đồ thị đặc biệt.
- Cây là đồ thị vô hướng, liên thông và không chứa chu trình.
- Cây có n đỉnh sẽ có đúng $n - 1$ cạnh.
- Hai đỉnh trong cây luôn có duy nhất một đường đi đơn.
- Cây không có chu trình nhưng nếu thêm vào một cạnh bất kỳ nối hai đỉnh không kề nhau, ta thu được duy nhất một chu trình.

Ví dụ cây



Các khái niệm quan trọng

- **Bậc của đỉnh**: Số cạnh nối với đỉnh đó.
- **Đỉnh lá**: Đỉnh có bậc bằng 1.
- **Đỉnh gốc**: Đỉnh được chọn làm điểm xuất phát trong cây.
- **Cây con**: Cây con của một cây T là một cây được tạo thành từ một đỉnh của T và tất cả các đỉnh con của nó.
- **Đường đi**: Một dãy các đỉnh liên tiếp nhau trong cây, không lặp lại đỉnh nào.
- **Chiều cao của cây**: Số cạnh trên đường đi dài nhất từ gốc đến một đỉnh lá.
- **Độ sâu của đỉnh**: Số cạnh trên đường đi từ gốc đến đỉnh đó.



Ứng dụng

- Quản lí dữ liệu phân cấp (hệ thống tập tin, cơ sở dữ liệu).
- Thuật toán AI: cây quyết định (Decsion tree).
- Thuật toán tìm kiếm của Google,... (Trie).
- Sơ đồ tổ chức công ty.
- Hệ thống phân cấp nhà nước.

Duyệt cây

- Thường dùng thuật toán DFS để duyệt cây.
- Dùng để tính chiều cao, độ sâu của câuy, đếm số đỉnh, tìm kiếm giá trị,...
- Dùng trong DP trên cây.
- Ý tưởng: Duyệt từ gốc, nếu các đỉnh kề không phải là cha của đỉnh hiện tại thì tiếp tục duyệt.

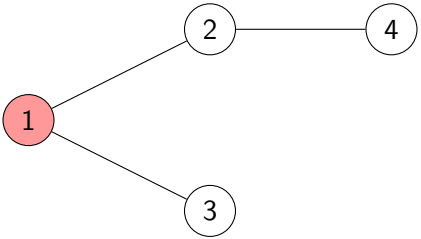
Độ phức tạp

$$O(V + E)$$

Code DFS trên cây

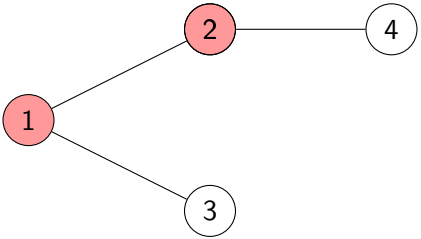
```
void dfs(int u, int parent)
{
    for(int v : adj[u])
    {
        if(v != parent)
        dfs(v, u);
    }
}
```

Mô phỏng DFS trên cây



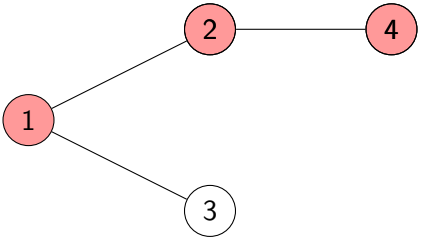
- Bắt đầu từ đỉnh 1

Mô phỏng DFS trên cây



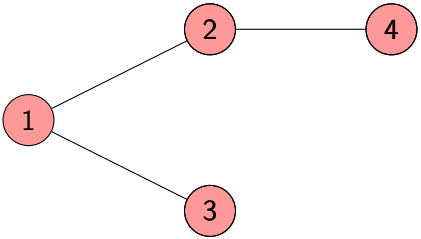
- Bắt đầu từ đỉnh 1
- Đi tới đỉnh 2

Mô phỏng DFS trên cây



- Bắt đầu từ đỉnh 1
- Đi tới đỉnh 2
- Đi tiếp tới đỉnh 4

Mô phỏng DFS trên cây



- Bắt đầu từ đỉnh 1
- Đi tới đỉnh 2
- Đi tiếp tới đỉnh 4
- Quay lui và thăm đỉnh 3

Thuật toán Disjoint Set Union (DSU)

- **Bài toán:**
  - Quản lý các tập hợp rời nhau
  - Hỗ trợ gộp tập và kiểm tra cùng tập
- **Ý tưởng cốt lõi:**
  - Mỗi tập hợp được biểu diễn bởi một cây
  - Mỗi phần tử lưu cha của nó
  - Nút gốc là đại diện của tập
- **Hai thao tác chính:**
  - **Find(u)**: Tìm nút gốc chứa u
  - **Union(u, v)**: Gộp hai tập bằng cách nối hai gốc
- **Tối ưu quan trọng:**
  - Path Compression: Nén đường đi khi Find
  - Union by Size/Rank: Gộp cây nhỏ vào cây lớn
- **Hiệu quả:** Độ phức tạp gần $O(1)$

Code DSU - khởi tạo

Ý tưởng:

- Mỗi đỉnh ban đầu là một tập riêng
- par[u] = -1 ⇒ u là gốc của tập
- sz[u] = 1 ⇒ kích thước tập bằng 1

Cài đặt:

```
ll par[N + 5], sz[N + 5], comps;

memset(par, -1, sizeof par);
fill(sz, sz + N + 2, 1);
```

Code DSU — Find và Union

Find(u)

- Tìm đại diện của tập chứa u
- Dùng Path Compression

Union(u, v)

- Tìm hai gốc của u và v
- Nếu khác tập thì gộp lại
- Gộp cây nhỏ vào cây lớn

```
ll find(ll u)
{
    return par[u] == -1 ? u : par[u] = find(par[u]);
}

void union(ll u, ll v)
{
    u = find(u);
    v = find(v);
    if (u == v) return;

    if (sz[u] < sz[v]) swap(u, v);
    par[v] = u;
    sz[u] += sz[v];
}
```

Mô phỏng DSU (Union-Find)

- Ban đầu: mỗi đỉnh là một tập riêng

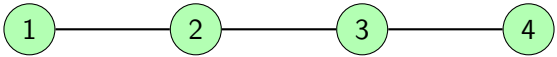
Mô phỏng DSU (Union-Find)

- Ban đầu: mỗi đỉnh là một tập riêng
- Union(1,2): gộp tập chứa 1 và 2

Mô phỏng DSU (Union-Find)

- Ban đầu: mỗi đỉnh là một tập riêng
- Union(1,2): gộp tập chứa 1 và 2
- Union(3,4): gộp tập chứa 3 và 4

Mô phỏng DSU (Union-Find)



- Ban đầu: mỗi đỉnh là một tập riêng
- Union(1,2): gộp tập chứa 1 và 2
- Union(3,4): gộp tập chứa 3 và 4
- Union(2,3): gộp hai tập lớn thành một

Một số thuật toán phổ biến khác trên cây

- **LCA (Lowest Common Ancestor)**: Tìm tổ tiên chung thấp nhất của hai đỉnh.
- **Centroid Decomposition**: Phân tách cây thành các phần nhỏ hơn để giải quyết bài toán phức tạp.
- **Heavy-Light Decomposition**: Chia cây thành các chuỗi nặng và nhẹ để tối ưu truy vấn trên cây.
- **Dynamic Programming trên cây**: Sử dụng DP để giải quyết các bài toán tối ưu hóa trên cây.
- **Euler Tour Technique**: Biến đổi cây thành một mảng để giải quyết các bài toán truy vấn trên cây.
- **Centroid Tree**: Xây dựng cây centroid để giải quyết các bài toán liên quan đến khoảng cách trên cây.

Tổng kết

- Cây là đồ thị vô hướng liên thông, không có chu trình
- Với n đỉnh luôn có đúng $n-1$ cạnh và duy nhất một đường đi giữa hai đỉnh
- Các thuật toán quan trọng: DFS/BFS, Tree DP, LCA
- Những bài toán về cây rất phổ biến trong lập trình thi đấu.
- Ứng dụng rộng rãi trong phân cấp dữ liệu và tối ưu mạng lưới

Cảm ơn các bạn đã lắng nghe!